

DeepSeek-V3.2 and Sparse Long Context

Deep Dive into DeepSeek — Chapter 11

Yin Yang

Foundation Books Project

The one rule of this chapter

DSA changes *which positions* reach the main attention path.

MLA changes *what is stored* for a retained position.

- Dense causal attention scores every earlier position: for length n , that is $n(n+1)/2$ score entries, so a longer context is quadratically more expensive.
- **DeepSeek Sparse Attention (DSA)** learns a lightning indexer that ranks prior positions, then top- k selection admits only the highest-ranked ones to the expensive MLA computation.
- Native Sparse Attention (NSA) is a neighboring research mechanism that motivates the design space; V3.2 ships DSA, not NSA.

The one rule of this chapter

DSA changes *which positions* reach the main attention path.

MLA changes *what is stored* for a retained position.

- Dense causal attention scores every earlier position: for length n , that is $n(n+1)/2$ score entries, so a longer context is quadratically more expensive.
- **DeepSeek Sparse Attention (DSA)** learns a lightning indexer that ranks prior positions, then top- k selection admits only the highest-ranked ones to the expensive MLA computation.
- Native Sparse Attention (NSA) is a neighboring research mechanism that motivates the design space; V3.2 ships DSA, not NSA.

Goal: follow selection from the indexer score through training, contrast it with MLA's compression, and see what an efficient selector still owes the task — recall, latency, and behavior.

1. Dense attention, and the axis DSA changes

1. Dense attention, and the axis DSA changes
2. NSA's design lesson, DSA's released mechanism

1. Dense attention, and the axis DSA changes
2. NSA's design lesson, DSA's released mechanism
3. Training the selector

1. Dense attention, and the axis DSA changes
2. NSA's design lesson, DSA's released mechanism
3. Training the selector
4. Sparse attention versus MLA

1. Dense attention, and the axis DSA changes
2. NSA's design lesson, DSA's released mechanism
3. Training the selector
4. Sparse attention versus MLA
5. Does the selector retain what the task needs?

1. Dense attention, and the axis DSA changes
2. NSA's design lesson, DSA's released mechanism
3. Training the selector
4. Sparse attention versus MLA
5. Does the selector retain what the task needs?
6. Where this leaves V3.2

Dense attention, and the axis DSA changes

Positions versus payload: two different knobs

Mechanism	Changes	Leaves alone
MLA	the stored payload $C_{r,\ell,t}$ per position	how many positions are scored

Positions versus payload: two different knobs

Mechanism	Changes	Leaves alone
MLA	the stored payload $C_{r,\ell,t}$ per position	how many positions are scored
DSA	the selected set $\mathcal{I}_{r,\ell,i}$ of positions	the payload shape once a position is selected

Positions versus payload: two different knobs

Mechanism	Changes	Leaves alone
MLA	the stored payload $C_{r,\ell,t}$ per position	how many positions are scored
DSA	the selected set $\mathcal{I}_{r,\ell,i}$ of positions	the payload shape once a position is selected

- A sparse attention mask gives query position i a selected set $\mathcal{I}_{r,\ell,i} = \{j : M_{i,j}^{(r,\ell)} = 1\}$ instead of the full causal prefix; its local density is $|\mathcal{I}_{r,\ell,i}|/i$.

Positions versus payload: two different knobs

Mechanism	Changes	Leaves alone
MLA	the stored payload $C_{r,\ell,t}$ per position	how many positions are scored
DSA	the selected set $\mathcal{I}_{r,\ell,i}$ of positions	the payload shape once a position is selected

- A sparse attention mask gives query position i a selected set $\mathcal{I}_{r,\ell,i} = \{j : M_{i,j}^{(r,\ell)} = 1\}$ instead of the full causal prefix; its local density is $|\mathcal{I}_{r,\ell,i}|/i$.
- At $n = 8$, dense causal attention has 36 score entries. A toy final row $\{1, 6, 7, 8\}$ has density $1/2$ — it succeeds only if the kept positions carry the needed evidence.

Positions versus payload: two different knobs

Mechanism	Changes	Leaves alone
MLA	the stored payload $C_{r,\ell,t}$ per position	how many positions are scored
DSA	the selected set $\mathcal{I}_{r,\ell,i}$ of positions	the payload shape once a position is selected

- A sparse attention mask gives query position i a selected set $\mathcal{I}_{r,\ell,i} = \{j : M_{i,j}^{(r,\ell)} = 1\}$ instead of the full causal prefix; its local density is $|\mathcal{I}_{r,\ell,i}|/i$.
- At $n = 8$, dense causal attention has 36 score entries. A toy final row $\{1, 6, 7, 8\}$ has density $1/2$ — it succeeds only if the kept positions carry the needed evidence.

Sparsity is arithmetic. Retrieval is a property of *which* four positions a trained selector keeps.

DeepSeek Sparse Attention, defined

Definition (DeepSeek Sparse Attention)

The V3.2 mechanism in which a lightning indexer ranks prior positions and fine-grained top- k selection admits only chosen latent-KV entries to the main attention path. The paper instantiates this selection under MLA in an MQA-style mode.

$$I_{t,s} = \sum_{j=1}^{H_I} w_{t,j}^I \text{ReLU}((q_{t,j}^I)^\top k_s^I), \quad S_t = \text{TopK}_k(I_{t,1:t})$$

Read the score as: sum over indexer heads of the head weight times the rectified dot product between the query-side indexer feature and the shared indexer key for candidate s . Top- k keeps the highest-scoring causal candidates as S_t .

DeepSeek Sparse Attention, defined

Definition (DeepSeek Sparse Attention)

The V3.2 mechanism in which a lightning indexer ranks prior positions and fine-grained top- k selection admits only chosen latent-KV entries to the main attention path. The paper instantiates this selection under MLA in an MQA-style mode.

$$I_{t,s} = \sum_{j=1}^{H_I} w_{t,j}^I \operatorname{ReLU}((q_{t,j}^I)^\top k_s^I), \quad S_t = \operatorname{TopK}_k(I_{t,1:t})$$

Read the score as: sum over indexer heads of the head weight times the rectified dot product between the query-side indexer feature and the shared indexer key for candidate s . Top- k keeps the highest-scoring causal candidates as S_t .

One shared indexer key per prior token, ranked once, reused across every indexer head.

[REPORTED: DeepSeek-V3.2 technical report]

**NSA's design lesson, DSA's
released mechanism**

NSA: three trained branches, one gated sum

Branch	Main object	Preserves
Compression	learned KV summary per block	coarse prefix coverage

NSA: three trained branches, one gated sum

Branch	Main object	Preserves
Compression	learned KV summary per block	coarse prefix coverage
Selection	original tokens, chosen blocks	fine detail, contiguous

NSA: three trained branches, one gated sum

Branch	Main object	Preserves
Compression	learned KV summary per block	coarse prefix coverage
Selection	original tokens, chosen blocks	fine detail, contiguous
Window	recent original tokens	local syntax, own projections

NSA: three trained branches, one gated sum

Branch	Main object	Preserves
Compression	learned KV summary per block	coarse prefix coverage
Selection	original tokens, chosen blocks	fine detail, contiguous
Window	recent original tokens	local syntax, own projections

Learned sigmoid gates combine the three outputs (need not sum to one); block-contiguous, group-shared selection is what makes the pattern executable rather than arbitrary token skipping.

NSA: three trained branches, one gated sum

Branch	Main object	Preserves
Compression	learned KV summary per block	coarse prefix coverage
Selection	original tokens, chosen blocks	fine detail, contiguous
Window	recent original tokens	local syntax, own projections

Learned sigmoid gates combine the three outputs (need not sum to one); block-contiguous, group-shared selection is what makes the pattern executable rather than arbitrary token skipping.

Reported: 27B-total/3B-active MoE, GQA, 64K sequences: up to $9.0\times$ forward / $6.0\times$ backward speedup, $11.6\times$ decode. [REPORTED: Native Sparse Attention paper]

NSA: three trained branches, one gated sum

Branch	Main object	Preserves
Compression	learned KV summary per block	coarse prefix coverage
Selection	original tokens, chosen blocks	fine detail, contiguous
Window	recent original tokens	local syntax, own projections

Learned sigmoid gates combine the three outputs (need not sum to one); block-contiguous, group-shared selection is what makes the pattern executable rather than arbitrary token skipping.

Reported: 27B-total/3B-active MoE, GQA, 64K sequences: up to $9.0\times$ forward / $6.0\times$ backward speedup, $11.6\times$ decode. [REPORTED: Native Sparse Attention paper]

Why V3.2 is not NSA: DSA ranks individual *positions*, not contiguous blocks, and trains in two stages against a released MLA implementation rather than end to end with its own kernels.

Training the selector

Two clocks: dense teacher, then sparse adaptation

Stage	What updates	Target
Dense warm-up	indexer only, rest frozen	KL to the head-aggregated dense-attention distribution p_t

$$\mathcal{L}_{\text{idx}}^{\text{dense}} = \sum_t D_{\text{KL}}(p_t \parallel \hat{p}_t), \quad \mathcal{L}_{\text{idx}}^{\text{sparse}} = \sum_t D_{\text{KL}}(p_t^S \parallel \hat{p}_t^S)$$

Two clocks: dense teacher, then sparse adaptation

Stage	What updates	Target
Dense warm-up	indexer only, rest frozen	KL to the head-aggregated dense-attention distribution p_t
Sparse training	indexer (KL) and main model (LM loss)	both distributions restricted & renormalized to S_t

$$\mathcal{L}_{\text{idx}}^{\text{dense}} = \sum_t D_{\text{KL}}(p_t \parallel \hat{p}_t), \quad \mathcal{L}_{\text{idx}}^{\text{sparse}} = \sum_t D_{\text{KL}}(p_t^S \parallel \hat{p}_t^S)$$

Two clocks: dense teacher, then sparse adaptation

Stage	What updates	Target
Dense warm-up	indexer only, rest frozen	KL to the head-aggregated dense-attention distribution p_t
Sparse training	indexer (KL) and main model (LM loss)	both distributions restricted & renormalized to S_t

$$\mathcal{L}_{\text{idx}}^{\text{dense}} = \sum_t D_{\text{KL}}(p_t \parallel \hat{p}_t), \quad \mathcal{L}_{\text{idx}}^{\text{sparse}} = \sum_t D_{\text{KL}}(p_t^S \parallel \hat{p}_t^S)$$

Reported scale: 2,048 selected KV tokens per query, about 2.1B warm-up tokens, about 943.7B sparse-training tokens. [REPORTED: DeepSeek-V3.2 technical report]

From concept to code: the indexer gates the main path

The reference model computes indexer scores, keeps the top- k indices, and adds an additive mask before the softmax — exactly the algebra above.

```
from DeepSeek-V3.2-Exp/inference/model.py

index_score = fp8_index(q_fp8, weights, self.k_cache[:bsz, :end_pos], ...)
topk_indices = index_score.topk(min(self.index_topk, end_pos), dim=-1)[1]
...
index_mask = torch.full((bsz, 1, end_pos), float("-inf"), ...) \
    .scatter_(-1, topk_indices, 0)
scores += index_mask.unsqueeze(2)
```

From concept to code: the indexer gates the main path

The reference model computes indexer scores, keeps the top- k indices, and adds an additive mask before the softmax — exactly the algebra above.

```
from DeepSeek-V3.2-Exp/inference/model.py

index_score = fp8_index(q_fp8, weights, self.k_cache[:bsz, :end_pos], ...)
topk_indices = index_score.topk(min(self.index_topk, end_pos), dim=-1)[1]
...
index_mask = torch.full((bsz, 1, end_pos), float("-inf"), ...) \
    .scatter_(-1, topk_indices, 0)
scores += index_mask.unsqueeze(2)
```

index_topk is k ; the scatter turns S_t into an additive mask, so everything outside the selected set is $-\infty$ before the softmax normalizes only the survivors. [INSPECTED: DeepSeek-V3.2-Exp inference/model.py]

Sparse attention versus MLA

Two mechanisms, two different objects

	Changed object	Failure mode
MLA	payload $C_{r,\ell,t}$ for each retained position	lost reconstructable key/value information

Two mechanisms, two different objects

	Changed object	Failure mode
MLA	payload $C_{r,\ell,t}$ for each retained position	lost reconstructable key/value information
DSA	selected set $\mathcal{I}_{r,\ell,i}$ available to query i	an answer-critical position omitted

Two mechanisms, two different objects

	Changed object	Failure mode
MLA	payload $C_{r,\ell,t}$ for each retained position	lost reconstructable key/value information
DSA	selected set $\mathcal{I}_{r,\ell,i}$ available to query i	an answer-critical position omitted

V3.2 does both: DSA selects, and the selected entries are MLA's compact (c_s^{KV}, k_s^R) pairs. Calling every efficiency trick “efficient attention” hides which quantity actually changed.

Two mechanisms, two different objects

	Changed object	Failure mode
MLA	payload $C_{r,\ell,t}$ for each retained position	lost reconstructable key/value information
DSA	selected set $\mathcal{I}_{r,\ell,i}$ available to query i	an answer-critical position omitted

V3.2 does both: DSA selects, and the selected entries are MLA's compact (c_s^{KV}, k_s^R) pairs. Calling every efficiency trick "efficient attention" hides which quantity actually changed. **MLA cannot buy back a position DSA drops; DSA cannot buy back information MLA's compression lost.**

Two mechanisms, two different objects

	Changed object	Failure mode
MLA	payload $C_{r,\ell,t}$ for each retained position	lost reconstructable key/value information
DSA	selected set $\mathcal{I}_{r,\ell,i}$ available to query i	an answer-critical position omitted

V3.2 does both: DSA selects, and the selected entries are MLA's compact (c_s^{KV}, k_s^R) pairs. Calling every efficiency trick "efficient attention" hides which quantity actually changed. **MLA cannot buy back a position DSA drops; DSA cannot buy back information MLA's compression lost. A**

combined-system hazard: the released path feeds the indexer a non-interleaved rotary convention while MLA uses the interleaved one. Shapes agree either way, so a mismatched layout corrupts indexer scores silently rather than raising an error. [INSPECTED: DeepSeek-V3.2-Exp inference/model.py]

**Does the selector retain what
the task needs?**

A dossier that moves, so position becomes the only variable



A dossier that moves, so position becomes the only variable



target

V4 distractor

A dossier that moves, so position becomes the only variable

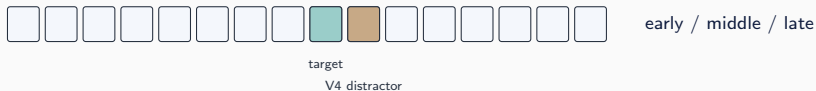


early / middle / late

target

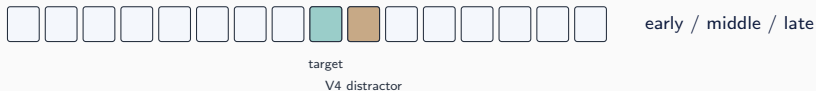
V4 distractor

A dossier that moves, so position becomes the only variable



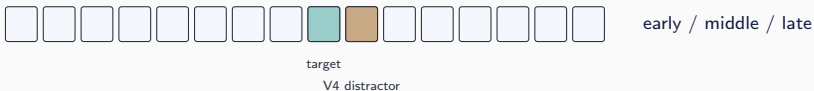
- Move one equal-token target–distractor unit among early, middle, and late slots; everything else — model, runtime, decoding, tool schema, scorers — stays fixed.

A dossier that moves, so position becomes the only variable



- Move one equal-token target–distractor unit among early, middle, and late slots; everything else — model, runtime, decoding, tool schema, scorers — stays fixed.
- Score target grounding, distractor rejection, chronology, variant-role handling, and final support *separately*.

A dossier that moves, so position becomes the only variable



- Move one equal-token target–distractor unit among early, middle, and late slots; everything else — model, runtime, decoding, tool schema, scorers — stays fixed.
- Score target grounding, distractor rejection, chronology, variant-role handling, and final support *separately*.

One reproducible middle-position failure refutes a universal retrieval guarantee for that placement — it does not by itself indict the indexer without an instrumented trace. [DERIVED: this chapter's paired-placement protocol]

Cost has (at least) three separable rows

$$C_{\text{DSA}}^{\text{prefill}} = \underbrace{C_{\text{indexer}}(n, d_I)}_{O(n^2 d_I)} + C_{\text{topk}}(n, k) + \underbrace{C_{\text{selected}}(n, k, d_A)}_{O(n k d_A)}$$

- The paper states the main path drops from quadratic full-context attention to $O(L_{\text{ctx}} k)$ — but the low-width indexer scan that ranks every candidate stays quadratic in n .

Cost has (at least) three separable rows

$$C_{\text{DSA}}^{\text{prefill}} = \underbrace{C_{\text{indexer}}(n, d_I)}_{O(n^2 d_I)} + C_{\text{topk}}(n, k) + \underbrace{C_{\text{selected}}(n, k, d_A)}_{O(n k d_A)}$$

- The paper states the main path drops from quadratic full-context attention to $O(L_{\text{ctx}} k)$ — but the low-width indexer scan that ranks every candidate stays quadratic in n .
- End-to-end latency additionally includes queueing, kernel choice, and tool calls; a lower selected-count estimate can still coexist with a slow kernel or a lost-quality answer.

Cost has (at least) three separable rows

$$C_{\text{DSA}}^{\text{prefill}} = \underbrace{C_{\text{indexer}}(n, d_I)}_{O(n^2 d_I)} + C_{\text{topk}}(n, k) + \underbrace{C_{\text{selected}}(n, k, d_A)}_{O(n k d_A)}$$

- The paper states the main path drops from quadratic full-context attention to $O(L_{\text{ctx}} k)$ — but the low-width indexer scan that ranks every candidate stays quadratic in n .
- End-to-end latency additionally includes queueing, kernel choice, and tool calls; a lower selected-count estimate can still coexist with a slow kernel or a lost-quality answer.

Four separate rows: analytical selected-count, kernel timing, end-to-end latency, and task quality. None of the first three implies the fourth. [REPORTED: DeepSeek-V3.2 technical report]

Post-training and tool use, in one picture

specialists
SFT+RL

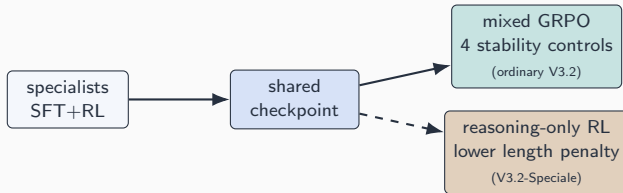
Post-training and tool use, in one picture



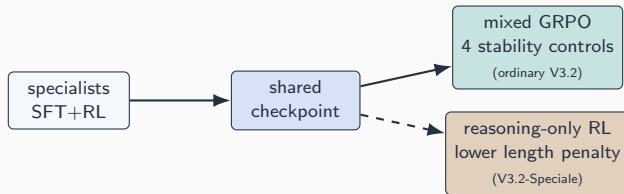
Post-training and tool use, in one picture



Post-training and tool use, in one picture

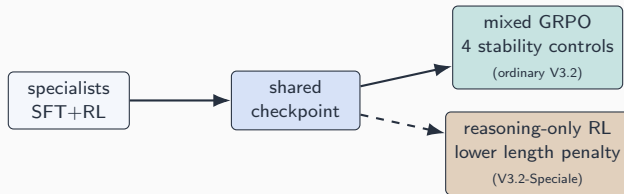


Post-training and tool use, in one picture



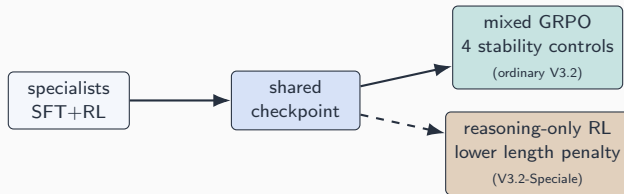
- GRPO's four controls (unbiased KL, off-policy masking, Keep Routing, Keep Sampling Mask) keep rollout and training comparisons valid at ten-plus-percent-of-pretraining post-training scale.

Post-training and tool use, in one picture



- GRPO's four controls (unbiased KL, off-policy masking, Keep Routing, Keep Sampling Mask) keep rollout and training comparisons valid at ten-plus-percent-of-pretraining post-training scale.
- A tool-capable path retains reasoning across a tool result but discards it on a new user turn — attention sparsity does not decide this; the client-side replay rule does.

Post-training and tool use, in one picture



- GRPO's four controls (unbiased KL, off-policy masking, Keep Routing, Keep Sampling Mask) keep rollout and training comparisons valid at ten-plus-percent-of-pretraining post-training scale.
- A tool-capable path retains reasoning across a tool result but discards it on a new user turn — attention sparsity does not decide this; the client-side replay rule does.

85,267 reported agentic prompts across four task families; the training signal comes from executing and scoring, not from the count alone. [REPORTED: DeepSeek-V3.2 technical report]

Where this leaves V3.2

What DSA does and does not buy

Buys

- A learned, query-dependent selected set — not a fixed local window or permanent global tokens.
- A main-attention path reduced to $O(L_{\text{ctx}}k)$.
- A trained-together indexer and sparse main path, with a dense teacher as the alignment target.

Does not buy

- A smaller per-position payload — that is MLA's axis, not DSA's.
- A retrieval guarantee for every placement of an answer-critical item.
- Automatic runtime speed: kernel fusion, memory traffic, and top- k implementation still require measurement.

What DSA does and does not buy

Buys

- A learned, query-dependent selected set — not a fixed local window or permanent global tokens.
- A main-attention path reduced to $O(L_{\text{ctx}}k)$.
- A trained-together indexer and sparse main path, with a dense teacher as the alignment target.

Does not buy

- A smaller per-position payload — that is MLA's axis, not DSA's.
- A retrieval guarantee for every placement of an answer-critical item.
- Automatic runtime speed: kernel fusion, memory traffic, and top- k implementation still require measurement.

DSA shortens the position list; MLA compresses what survives on that list. V4 changes what a selected index even names — Chapter 13.

Concept-to-code map for the chapter

- **indexer state, scoring, top- k** $\rightarrow$ `DeepSeek-V3.2-Exp/inference/model.py`

Concept-to-code map for the chapter

- **indexer state, scoring, top- k** $\rightarrow$ `DeepSeek-V3.2-Exp/inference/model.py`
- **fp8 weighted-ReLU indexer scoring kernel** $\rightarrow$ `DeepSeek-V3.2-Exp/inference/kernel.py`

Concept-to-code map for the chapter

- **indexer state, scoring, top- k** → `DeepSeek-V3.2-Exp/inference/model.py`
- **fp8 weighted-ReLU indexer scoring kernel** → `DeepSeek-V3.2-Exp/inference/kernel.py`
- **indexer shape and top- k budget** → `DeepSeek-V3.2-Exp/inference/config_671B_v3.2.json`

Concept-to-code map for the chapter

- **indexer state, scoring, top- k** → `DeepSeek-V3.2-Exp/inference/model.py`
- **fp8 weighted-ReLU indexer scoring kernel** → `DeepSeek-V3.2-Exp/inference/kernel.py`
- **indexer shape and top- k budget** → `DeepSeek-V3.2-Exp/inference/config_671B_v3.2.json`
- **sparse decode kernel interface** → `FlashMLA/csrc/api/sparse_decode.h`

Concept-to-code map for the chapter

- **indexer state, scoring, top- k** → `DeepSeek-V3.2-Exp/inference/model.py`
- **fp8 weighted-ReLU indexer scoring kernel** → `DeepSeek-V3.2-Exp/inference/kernel.py`
- **indexer shape and top- k budget** → `DeepSeek-V3.2-Exp/inference/config_671B_v3.2.json`
- **sparse decode kernel interface** → `FlashMLA/csrc/api/sparse_decode.h`
- **sparse prefill kernel interface** → `FlashMLA/csrc/api/sparse_fwd.h`

Concept-to-code map for the chapter

- **indexer state, scoring, top- k** → `DeepSeek-V3.2-Exp/inference/model.py`
- **fp8 weighted-ReLU indexer scoring kernel** → `DeepSeek-V3.2-Exp/inference/kernel.py`
- **indexer shape and top- k budget** → `DeepSeek-V3.2-Exp/inference/config_671B_v3.2.json`
- **sparse decode kernel interface** → `FlashMLA/csrc/api/sparse_decode.h`
- **sparse prefill kernel interface** → `FlashMLA/csrc/api/sparse_fwd.h`

Rank positions, select the top- k , read only what survives.

Next: V4 changes what a selected index names when compression and selection are combined more tightly — Chapter 13.